

Smart Contract Audit Report

Looputo Finance

24 JUL 2026

Numen Cyber Labs - Security Services

Table of Content

1 Executive Summary 3

Methodology 3

2 Findings Overview 4

2.1 Project Info 4

2.2 Audit Scope 4

2.3 Findings Summary 6

2.4 Key Findings 6

3 Detailed Description of Findings 7

3.1 M-01 Unbounded router execution surface in swap integration 8

3.2 M-02 Factory-owned pool caps cannot be updated 10

3.3 M-03 Zero aggregate shares can block position closure 12

3.4 L-01 Unvalidated LTV thresholds can bypass risk limits or freeze pools 14

3.5 L-02 Zero output permitted during releverage 16

3.6 L-03 Releverage bypasses minimum share size 17

3.7 L-04 Last batch entry receives unrelated remainder 18

3.8 L-05 Zero debt price freezes collateral entry 19

3.9 L-06 Silent immediate configuration changes 20

4 Disclaimer 21

1 Executive Summary

A security assessment of the Looputo Finance smart contract implementation was conducted from 06/24/2026 to 07/24/2026 by three auditors. The assessment evaluated the in-scope contracts for potential security issues, and this report provides a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations to resolve the underlying technical issues.

Nine findings are reported: three Medium and six Low severity. Three Medium and four Low findings have been fixed; the remaining two Low findings are acknowledged. Reported issues were validated with executable Foundry proof-of-concept tests against a mainnet fork of the audited commit.

Methodology

The review began by reasoning about the protocol's business logic — the position lifecycle, share accounting, flash-loan flows, and privileged configuration surface — to check for gaps between the intended logic and the implementation. The code was then examined for known security flaws and attack vectors, with particular attention to the trust boundaries around approved aggregators, oracles, and owner/keeper roles.

Findings are classified by combining the potential impact of an issue with its likelihood of being exploited. Likelihood and impact are categorized into three ratings: High, Medium and Low. Severity is determined by likelihood and impact, and is classified into four categories accordingly: Critical, High, Medium and Low, as shown in Table 1.1. Each issue is nevertheless evaluated on a case-by-case basis and may not strictly follow the table.

Impact / Likelihood	Low	Medium	High
Low	Low	Low	Low
Medium	Low	Medium	Medium
High	Medium	High	High
Critical	High	Critical	Critical

Table 1.1: Severity classification by impact and likelihood

2 Findings Overview

2.1 Project Info

Name	Looputo Finance
Repository	https://github.com/looputo-finance/Looputo-Finance
Commit	`70b1298`
Fix Commit	`3a37647`
Language	Solidity
Timeline	06/24/2026 to 07/24/2026
№ of Auditors	3

Table 2.1: Project information

Looputo Finance is an automated leveraged-position protocol. It allows users to deposit assets into a shared pool and set their own leverage, while keepers monitor the pool to mitigate liquidation and negative-carry risks.

Each pool maintains per-user collateral and debt share positions while supplying collateral and borrowing debt through an Aave V3, Spark, or Morpho Blue lending adapter. Position operations — open, add/reduce leverage, add/reduce equity, and close — are executed atomically using Morpho flash loans combined with swaps through owner-approved DEX aggregators.

Pools are deployed as beacon proxies through an owner-controlled `Factory`. The initializer configures the collateral and debt assets, lending-market addresses, LTV limits, deposit cap, fees, approved aggregators, keeper, and owner. A designated keeper can rebalance pools in batches via `deleverage` and `releverage`.

2.2 Audit Scope

File	Description
`src/LoopPool.sol`	Abstract common leveraged-position and flash-loan engine
`src/AaveV3LoopPool.sol`	Aave V3-backed pool adapter
`src/SparkLoopPool.sol`	Spark-backed pool adapter

`src/MorphoLoopPool.sol`	Morpho Blue-backed pool adapter
`src/Factory.sol`	Owner-controlled beacon-proxy deployment and pool-configuration forwarder
`src/LoopPoolBeacon.sol`	`UpgradeableBeacon` wrapper

Table 2.2: Contracts in scope

Third-party dependencies (OpenZeppelin, Morpho Blue, Aave V3, Spark, and external swap aggregators) were treated as trusted and reviewed only at their integration boundaries.

2.3 Findings Summary

Severity	Found	
Critical	0	
High	0	
Medium	3	
Low	6	

Table 2.3: Findings by severity

2.4 Key Findings

ID	Severity	Findings Title	Status
M-01	**Medium**	Unbounded router execution surface in swap integration	**Fixed**
M-02	**Medium**	Factory-owned pool caps cannot be updated	**Fixed**
M-03	**Medium**	Zero aggregate shares can block position closure	**Fixed**
L-01	**Low**	Unvalidated LTV thresholds can bypass risk limits or freeze pools	**Acknowledged**
L-02	**Low**	Zero output permitted during releverage	**Fixed**
L-03	**Low**	Releverage bypasses minimum share size	**Fixed**
L-04	**Low**	Last batch entry receives unrelated remainder	**Acknowledged**
L-05	**Low**	Zero debt price freezes collateral entry	**Fixed**
L-06	**Low**	Silent immediate configuration changes	**Fixed**

Table 2.4: Key audit findings

3 Detailed Description of Findings

This chapter provides a full description of each identified issue, the associated risk rating, the validated impact, and detailed recommendations to resolve the underlying technical issue. Findings are ordered by severity.

3.1 M-01 Unbounded router execution surface in swap integration

Severity: Medium **Status:** Fixed

`src/LoopPool.sol (initialization, setAggregator, onMorphoFlashLoan), src/Factory.sol`

Description

`LoopPool` gives every approved aggregator a standing maximum allowance for `COLL_TOKEN` and `DEBT_TOKEN`, then passes caller- or keeper-selected `swapData` to that address through a low-level call in the flash-loan callback. The allowlist authenticates only the router address: it does not bind a selector, token pair, input budget, recipient, or nested execution behavior, while the callback validates only a minimum delta for one output token. A generic, compromised, or overly permissive approved router can therefore use its allowances to move unrelated pool balances while arranging sufficient measured output for settlement to succeed. This applies across the open, add, close, leverage, reduce, and keeper paths.

```
// __LoopPool_init — unlimited standing approvals for every aggregator
for (uint256 i = 0; i < _aggregators.length; i++) {
    approvedAggregator[_aggregators[i]] = true;
    COLL_TOKEN.forceApprove(_aggregators[i], type(uint256).max);
    DEBT_TOKEN.forceApprove(_aggregators[i], type(uint256).max);
}

// onMorphoFlashLoan — arbitrary calldata, only output delta is checked
(bool success, ) = aggregatorRouter.call(swapData);
require(success, "aggregator swap failed");
uint256 receivedAmount = collTokenBalanceAfter - collTokenBalanceBefore;
require(receivedAmount >= amount, "Swap exceeds defined slippage");
```

Root Cause

Standing maximum approvals to aggregators are combined with unrestricted router calldata and one-sided output-delta checks, rather than an operation-bounded, swap-specific token-flow policy.

Impact

A position user, or a keeper on the keeper-only paths, can trigger an approved router with a transfer, sweep, multicall, or generic-execution surface to extract residual or transient pool collateral and debt tokens to an attacker-controlled recipient. The pool can still repay the flash loan through its normal borrow or repayment flow, leaving the value loss or additional lending debt borne by pooled positions. Exploitation depends on an approved router having the relevant callable capability or being compromised, but the affected assets can belong to other users.

Proof of Concept

A mainnet-fork Foundry test approves a router exposing a generic execution surface, seeds the pool with residual debt tokens, and opens a position whose swap calldata instructs the router to both satisfy the required output delta and sweep the residual balance via its standing allowance:

```
function test_unboundedApprovedRouterCalldataDrainsResidualPoolDebtToken() public {
    assertEq(IERC20(WETH).allowance(address(pool), address(evil)),
             type(uint256).max, "standing WETH approval");
    ...
    // position opens successfully AND the residual balance is stolen
    assertEq(uint256(status), uint256(ILoopPool.PositionStatus.Active));
    assertEq(IERC20(WETH).balanceOf(attacker),
             attackerWethBefore + residualDebtToken, "attacker stole residual WETH");
}
```

Recommendations

Remove standing router allowances. Grant only an operation-scoped allowance for the expected input token and amount inside the authenticated flash-loan callback, and revoke it afterwards. Re-check router approval on keeper paths, and reject lending-core addresses (e.g. the Morpho or market contracts) as approvable aggregators.

3.2 M-02 Factory-owned pool caps cannot be updated

Severity: Medium **Status:** Fixed

`src/Factory.sol (L50-88), src/LoopPool.sol (L91-115, L1114-1120, L1348-1357)`

Description

Factory-administered pools can be initialized with the `Factory` as their owner, while `LoopPool.setCap` remains callable only by that owner. The Factory exposes forwarding methods for every other owner-only configuration operation (`setSystemTargetLTV`, `setPositionMaxLTV`, `setFeeConfiguration`, `setKeeper`, `setFlashloanFeePercentage`, `setAggregator`), but its forwarding surface contains no method that invokes `setCap`. The external Factory owner therefore cannot satisfy `onlyOwner` on the pool and has no Factory-mediated route to adjust `cap`: the configured limit is effectively immutable for Factory-owned pools despite `setCap` existing on the implementation.

```
// LoopPool — cap setter restricted to the pool owner (the Factory)
function setCap(uint256 newCap) external onlyOwner {
    require(newCap != cap, "New cap must be different from the current one");
    cap = newCap;
}

// LoopPool — deposits revert once net equity exceeds the frozen cap
require(poolNetEquity <= cap, "Deposit amount exceeds cap");
```

Root Cause

The Factory omits an owner-gated forwarding function for `LoopPool.setCap` even though it can own and administer initialized pools.

Impact

A mistakenly low cap can immediately prevent users from opening positions or adding equity, and a reached cap blocks those operations until withdrawals reduce `poolNetEquity`. Operators also cannot reduce the limit during an incident or increase capacity as conditions change without transferring ownership, upgrading, or replacing the pool.

Proof of Concept

A mainnet-fork test deploys a Factory-owned pool with a tight cap, verifies that other Factory forwarding functions work, and demonstrates that no route can change the cap while further deposits revert:

```
vm.prank(owner);
vm.expectRevert();
tightPool.setCap(tightCap + 100 ether);            // direct call: not the owner

(bool ok, ) = address(factory).call(
    abi.encodeWithSignature("setCap(address,uint256)", address(tightPool), newCap));
assertFalse(ok, "Factory exposes no setCap forwarding route");
```

```
vm.expectRevert(bytes("Deposit amount exceeds cap"));
pool.openPosition(params);                          // second user blocked
```

Recommendations

Add an owner-gated `setCap(address pool, uint256 newCap)` forwarding function to the Factory, mirroring the existing configuration forwarders, or assign pool ownership to an address that can call `setCap` directly.

3.3 M-03 Zero aggregate shares can block position closure

Severity: Medium **Status:** Fixed

`src/LoopPool.sol (closePosition L621-685, deleverage L701-763), src/libraries/SharesLib.sol (L31-45)`

Description

`closePosition` converts the caller's shares with `toAssetsDown` and `toAssetsUp` before it deletes the position, but neither conversion handles a zero aggregate-share denominator. The keeper path can repay all aggregate debt while leaving collateral and a `Deleveraged` position: `deleverage` subtracts the computed debt shares from `totalDebtShares` and distributes the result without requiring a nonzero residual aggregate. The caller then reaches `toAssetsUp(userDebtShares, totalDebtShares, totalDebtAmount)` with `totalDebtShares == 0`, and `SharesLib.toAssetsUp` divides by that value even when `userDebtShares == 0`. A full collateral unwind can analogously leave a non-`Inactive` zero-share position and make the collateral conversion divide by `totalCollShares == 0`.

```
// SharesLib — both conversions divide by totalShares unconditionally
function toAssetsDown(uint256 shares, uint256 totalShares, uint256 totalAssets)
    internal pure returns (uint256)
{
    return shares.mulDivDown(totalAssets, totalShares);
}

function toAssetsUp(uint256 shares, uint256 totalShares, uint256 totalAssets)
    internal pure returns (uint256)
{
    return shares.mulDivUp(totalAssets, totalShares);
}
```

Root Cause

The close conversion path assumes `totalCollShares` and `totalDebtShares` are nonzero, while the keeper unwind path can legitimately reduce either aggregate to zero without deleting or specially handling the remaining positions.

Impact

A user whose position has been fully deleveraged or unwound cannot invoke the normal close path to clear the position or retrieve remaining collateral; the call reverts on a division-by-zero panic before settlement. This can strand collateral and leave stale non-`Inactive` position records that also prevent reopening under the same address.

Exploit Scenario

The keeper runs a full unwind batch that drives `totalDebtShares` (or `totalCollShares`) to zero while listed positions remain in the `Deleveraged` state. Any affected user's subsequent `closePosition` reverts inside the

share-to-asset conversion, and the position cannot be closed until aggregate shares become nonzero again through another path.

Recommendations

Guard the zero-denominator case in `SharesLib.toAssetsDown`/`toAssetsUp` (returning zero assets for zero total shares), and/or have the keeper unwind path delete or specially settle positions whenever it reduces an aggregate share total to zero.

3.4 L-01 Unvalidated LTV thresholds can bypass risk limits or freeze pools

Severity: Low **Status:** Acknowledged

`src/LoopPool.sol (init & LTV setters), src/SparkLoopPool.sol (L33-53, L245-324), src/AaveV3LoopPool.sol, src/MorphoLoopPool.sol`

Description

`__LoopPool_init` and the owner LTV setters store `systemTargetLTV` and `positionMaxLTV` as unconstrained integers, with setters checking only that a value changes. The concrete adapters consume these values in adapter-specific arithmetic: Aave V3 and Spark use `percentDivCeil` with a `1e4` percentage factor, while the Morpho path uses WAD-scaled LTV multiplication. The repository's own Spark tests demonstrate the unit mismatch by accepting `0.93 * 1e18` even though the Spark configuration is otherwise expressed as `9100`; using that WAD value in percentage division makes the collateral requirement far weaker than a 93% limit. Zero is likewise accepted even though Aave/Spark percentage division reverts on a zero divisor, while a Morpho zero threshold rejects positive-debt positions.

```
function setSystemTargetLTV(uint256 newSystemTargetLTV) external onlyOwner {
    require(newSystemTargetLTV != systemTargetLTV,
        "System target LTV must be different from the current one");
    systemTargetLTV = newSystemTargetLTV;   // no range or scale validation
}
```

Root Cause

`LoopPool` stores `systemTargetLTV` and `positionMaxLTV` without enforcing the nonzero, bounded, adapter-specific units required by downstream LTV checks.

Impact

A privileged owner or factory misconfiguration can set an oversized or incorrectly scaled threshold that admits positions beyond the pool's intended LTV buffer, increasing liquidation and bad-debt exposure subject to the lending venue's independent checks. Setting either threshold to zero can make debt-bearing position lifecycle and keeper operations revert; because `closePosition` checks user LTV before settlement, existing users can be unable to exit until a privileged repair is made.

Proof of Concept

A mainnet-fork test opens a real Aave-backed position, then sets `positionMaxLTV` to zero and shows that the user's `closePosition` reverts on the zero-divisor LTV check, freezing exit until the owner restores a valid value:

```
function test_poc_zeroPositionMaxLTVFreezesUserCloseUntilPrivilegedRepair() public {
    // active position with real Aave collateral and debt ...
    factory.setPositionMaxLTV(address(pool), 0);
    vm.expectRevert();          // percentage division by zero
```

```
    pool.closePosition(closeParams);
}
```

Recommendations

Validate each threshold on every initialization and update path against the scale and safe nonzero range for the selected adapter (e.g. `0 < value ≤ 1e4` for Aave/Spark percentage units), so that neither unit mismatches nor out-of-range values can be stored.

3.5 L-02 Zero output permitted during releverage

Severity: Low **Status:** Fixed

`src/LoopPool.sol (releverage L752-806, onMorphoFlashLoan add path)`

Description

`releverage` passes the keeper-supplied `minReceivedAmount` as the flash-loan callback's required collateral output, but zero is accepted and there is no requirement that the swap produce a positive amount. A low-level call to an EOA or a no-op router can return success while leaving the collateral balance unchanged; the callback then accepts `receivedAmount == 0` and still invokes `_supplyCollateral(receivedAmount)` followed by `_borrowDebt(assets)`. The releverage path therefore does not enforce, at the pool level, that flash-loaned debt is exchanged for newly received collateral before increasing the lending-market debt; the aggregate LTV check may permit the operation whenever the pool has existing collateralization headroom.

```
// releverage — keeper's minReceivedAmount is forwarded unchecked (0 allowed)
_executeFlashloan(
    true,
    params.minReceivedAmount,
    0,
    params.aggregatorRouter,
    params.data,
    params.flashloanDebtAmount
);

// callback — only receivedAmount >= amount is enforced; 0 >= 0 passes
uint256 receivedAmount = collTokenBalanceAfter - collTokenBalanceBefore;
require(receivedAmount >= amount, "Swap exceeds defined slippage");
```

Root Cause

The additive releverage path treats zero as a valid minimum collateral output and has no post-swap requirement that the collateral delta or minted shares be positive.

Impact

A keeper can mark a batch as releveraged without obtaining swap output, potentially increasing shared lending debt against existing users' collateral and distributing debt shares to the listed positions. Even where a concrete lending adapter rejects a literal zero-amount supply, any dust output satisfies the pool-level check, so the intended positive-output invariant rests on adapter-specific zero-amount behavior rather than on pool-level validation.

Recommendations

Require a strictly positive `minReceivedAmount` on `releverage`, and additionally require that the resulting collateral delta and minted shares are positive before distributing releveraged shares.

3.6 L-03 RELEVERAGE BYPASSES MINIMUM SHARE SIZE

Severity: Low **Status:** Fixed

`src/LoopPool.sol (releverage L752-806, _distributeReleverageShares L1297-1346)`

Description

Unlike opening and user leverage/equity additions — which each enforce `MIN_SHARE_DELTA` on every minted or burned amount — `releverage` never checks `MIN_SHARE_DELTA` for either the aggregate mint or each participant's allocation. Proportional `mulDivDown` can therefore produce zero or dust collateral/debt shares for an entry, while `_distributeReleverageShares` still overwrites that record's state and unconditionally sets its status to `Active`. A positive batch can consequently reactivate a position, or create a state transition whose assigned operation is below the configured minimum share size. The final system and per-listed-user LTV checks do not restore the missing minimum-share invariant.

```
// _distributeReleverageShares — no MIN_SHARE_DELTA check on the allocation,
// and the entry is reactivated regardless of the allocated amount
position.collShares = newCollShares;
position.debtShares = newDebtShares;

position.status = PositionStatus.Active;
```

Root Cause

The keeper distribution path bypasses both aggregate and per-entry minimum-share validation and treats metadata mutation as valid even when the allocation rounds to zero or dust.

Impact

Keeper batches can create dust or zero-delta active records that do not satisfy the minimum-operation assumptions used by normal user flows. Repeated operational batches can accumulate inconsistent lifecycle metadata and dust positions, creating avoidable close/modify liveness and accounting edge cases.

Recommendations

Enforce `MIN_SHARE_DELTA` on the aggregate releverage mint and on each per-entry allocation, and skip or reject entries whose allocation rounds to zero instead of mutating their status.

3.7 L-04 Last batch entry receives unrelated remainder

Severity: Low **Status:** Acknowledged

`src/LoopPool.sol (_distributeDeleveragedShares L1244-1295, _distributeReleverageShares L1297-1346)`

Description

In the keeper batch-distribution helpers, every non-final entry computes its share deltas proportionally — `params.collAmounts[i]` divided by `params.swapAmount` for deleverage, and `params.debtAmounts[i]` divided by `params.flashloanDebtAmount` for releverage — but the final entry receives the entire unallocated remainder regardless of its own weight. If the final entry has zero weight, or is a duplicate whose final occurrence has zero weight, it can receive a nonzero collateral and debt delta created solely by floor rounding in earlier entries. No uniqueness or final-weight consistency check prevents this allocation, and the collateral and debt remainders are computed independently, so a participant can be debited or credited in proportions unrelated to its listed amount.

```
// _distributeDeleveragedShares — last entry absorbs the full remainder
uint256 deleveragedCollShares = i != len - 1
    ? params.collAmounts[i].mulDivDown(
        cache.totalDeleveragedCollShares,
        params.swapAmount
    )
    : cache.totalDeleveragedCollShares - alreadyDeleveragedCollShares;
```

Root Cause

Rounding reconciliation is implemented as an unconditional last-entry remainder rather than as a remainder assigned to a participant with a validated positive weight. The batch schema also permits duplicate addresses and zero weights.

Impact

A listed user can lose additional collateral and debt shares beyond the amount implied by its batch weight, or the transaction can revert when the forced remainder exceeds that user's shares. In a large batch, accumulated floor rounding can make the forced remainder materially larger than a single-unit dust discrepancy.

Recommendations

Validate batch inputs (unique addresses, positive weights, weights summing to the batch total) and assign any rounding remainder to a participant with a validated positive weight, bounding it to the rounding error rather than the full unallocated amount.

3.8 L-05 Zero debt price freezes collateral entry

Severity: Low **Status:** Fixed

`src/AaveV3LoopPool.sol (convertCollToDebt), src/LoopPool.sol (L167-185, L445-463)`

Description

The Aave adapter's `convertCollToDebt` performs an unchecked division by the oracle's debt-token price. `IPriceOracle.getAssetPrice` returns an unconstrained `uint256`, and the converter does not reject a zero price before dividing. Because `_increaseNetEquity` is called before the flash-loan path, any collateral-funded `openPosition` or `addPositionEquity` reverts as soon as the provider oracle returns zero for the configured debt asset, regardless of the user's swap data or slippage parameters.

```
// openPosition — the converter runs before the flash-loan operation
inToken.safeTransferFrom(msg.sender, address(this), params.amount);
...
_increaseNetEquity(params.isColl, params.amount - fee);
// -> convertCollToDebt: collValue / debtPrice with no zero-price check
```

Root Cause

The adapter treats an external oracle result as a valid divisor without checking that `debtPrice` is nonzero; neither the interface nor the integration enforces a nonzero-price invariant.

Impact

A zero or unavailable debt price temporarily freezes all collateral-funded opens and collateral-funded equity additions for the Aave pool. Transactions revert atomically, so the immediate consequence is liveness loss rather than fund loss; debt-token-funded opens are unaffected because they bypass `convertCollToDebt`.

Proof of Concept

```
function test_zeroDebtPriceFreezesCollateralFundedOpenPosition() public {
    assertEq(IPriceOracle(oracle).getAssetPrice(WETH), 0,
        "precondition: Aave oracle returns zero WETH/debt price");
    vm.expectRevert(stdError.divisionError);
    pool.openPosition(params);   // collateral-funded open remains frozen
}
```

Recommendations

Validate that the oracle debt price is nonzero before using it as a divisor, replacing the unchecked arithmetic panic with an explicit, descriptive revert at the external-data trust boundary.

3.9 L-06 Silent immediate configuration changes

Severity: Low **Status:** Fixed

`src/LoopPool.sol (L138-185, L1098-1183)`

Description

The owner setters change risk thresholds, cap, fees, keeper, flash-loan configuration, and router allowances immediately, but emit no configuration events. Existing positions are evaluated using the newly stored values on their next operation, and router approval changes take effect immediately for all users. Because the interface defines events only for position operations and not for these administrative changes, monitoring systems cannot reconstruct the configuration timeline from the pool's event stream. This is especially significant for threshold and router changes that can alter whether an active position can be modified or closed.

```
function setCap(uint256 newCap) external onlyOwner {
    require(newCap != cap, "New cap must be different from the current one");
    cap = newCap;               // immediate effect, no event emitted
}
```

Root Cause

Administrative state changes lack timelock and event instrumentation despite directly affecting active positions and token allowances.

Impact

An owner can silently alter active-position economics and execution capability between a user's transaction submission and execution, while event-driven monitors receive no canonical change notification. Users and keepers may act on stale thresholds, fee assumptions, or router availability, and incident responders lose an auditable configuration trail.

Proof of Concept

A fork test records all logs while the owner disapproves the only router between a user's submission and execution; the user's already-submitted `openPosition` reverts with `"Aggregator not approved"` and no configuration event is found in the recorded log stream.

Recommendations

Emit dedicated events for every administrative configuration change (LTV thresholds, cap, fees, keeper, flash-loan fee, and aggregator approvals), and consider a timelock for parameters that affect active positions.

4 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time-, resource- and expertise-bound effort to find as many vulnerabilities as possible. We cannot guarantee 100% security after the review, or even that the review will find any problems with your smart contracts. Subsequent security reviews, a public bug bounty program and on-chain monitoring are strongly recommended.

This report is not investment advice, nor an endorsement of the Looputo Finance protocol or its team. It reflects the state of the codebase at the reviewed commit only; any change to the code after the review invalidates the conclusions of this report for the changed portions.